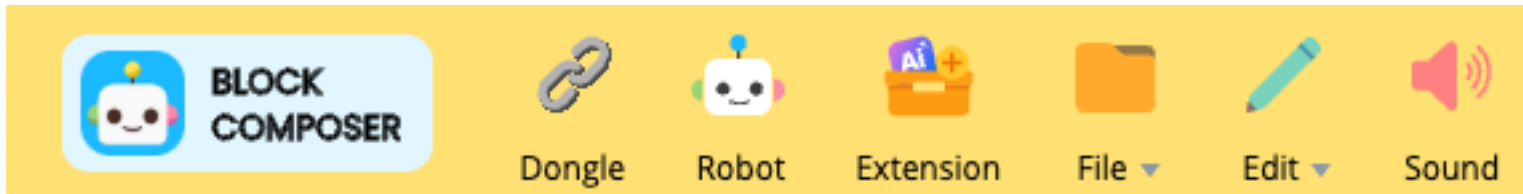


Components and Instructions

Top-Menu



The **RobomationLAB Top Menu** provides quick access to frequently used features such as connecting dongles and robots, saving and loading files, and running code.

Below is a detailed explanation of each menu item in order.

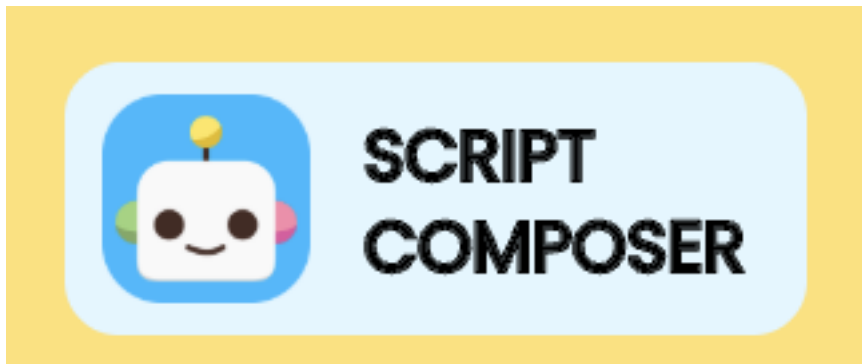
Logo



This is the RobomationLAB logo.

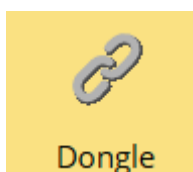
Clicking the logo takes you to the RobomationLAB main homepage.

When the **Block Coding Editor** is active, the logo text is displayed as **Block Composer**.



When the **Python / JavaScript Editor** is active, the logo text is displayed as **Script Composer**.

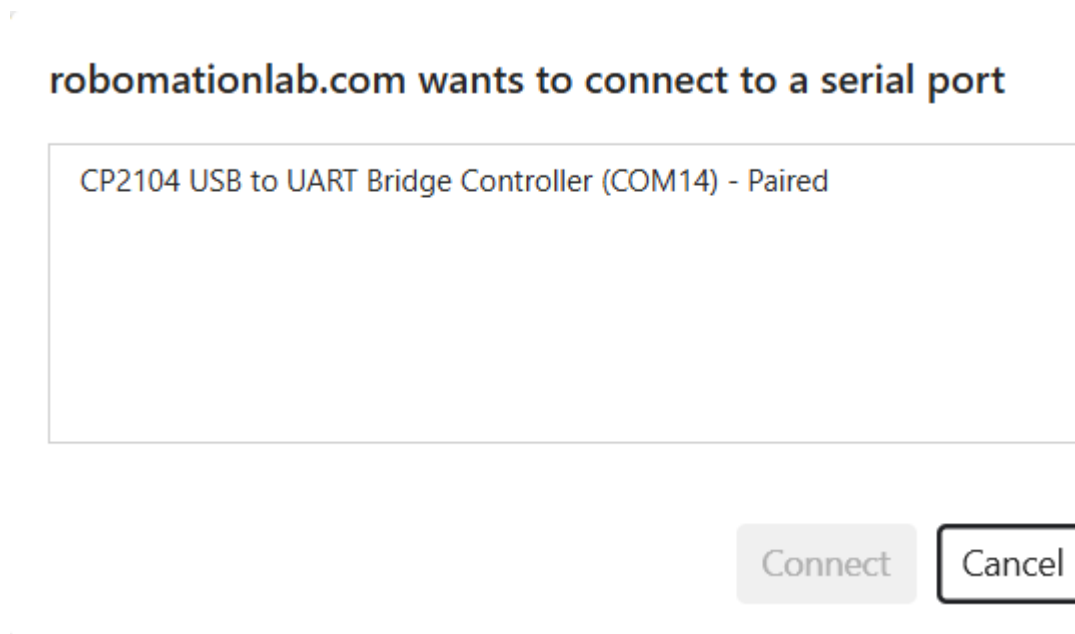
Dongle



You can **search for and connect a dongle used to communicate with robots and connect it to the program.**

To control a robot in the program, the dongle used to communicate with the robot must first be connected to the program.

This process is called **pairing**.



When you click the Dongle button, a list of dongles available on the current PC is displayed.

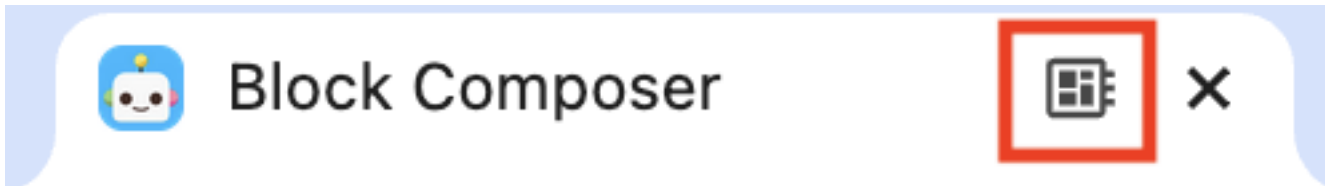
Select the desired dongle from the list and click the **Connect** button to connect the dongle to the program.

Checking Dongle Connection Status

Once a dongle has been connected to the program, it will be automatically reconnected the next time the program is used.

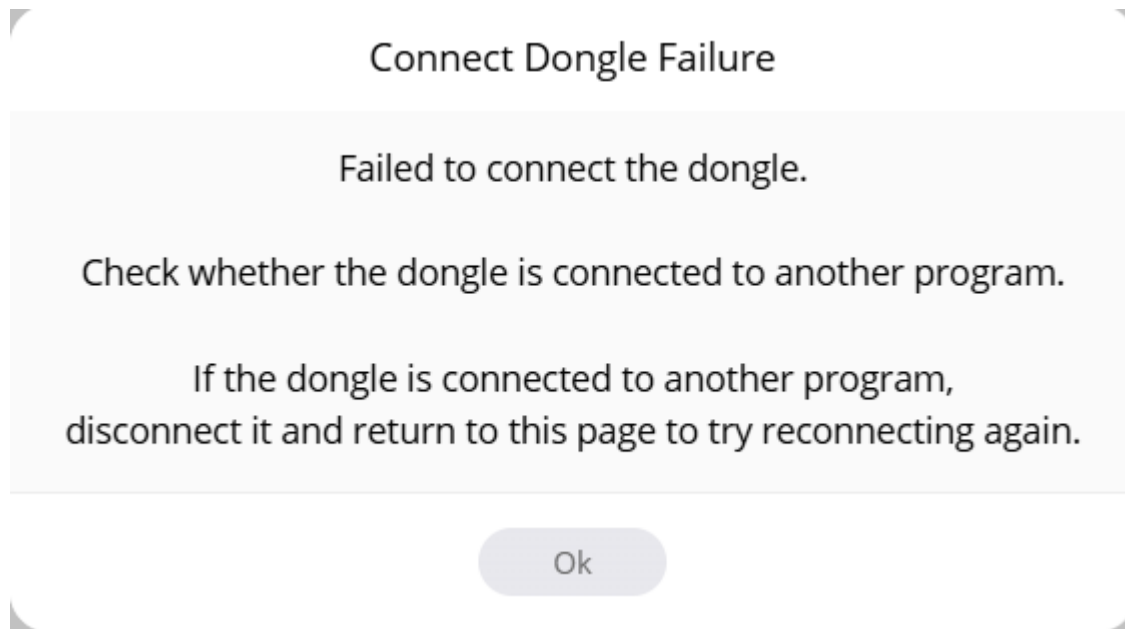


When the dongle is successfully connected to the program, the icon changes to **light blue**.



If the browser tab displays an icon like the one shown below, you can confirm that the dongle is connected.

Caution: Notice

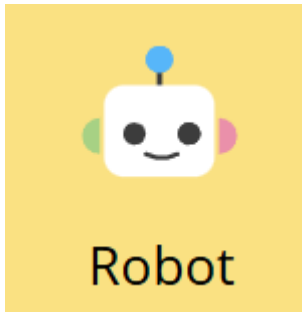


If the dongle is already connected to another program or another browser page, it cannot be connected to this program.

In this case, disconnect the dongle from the other program or page, then return to this page and try

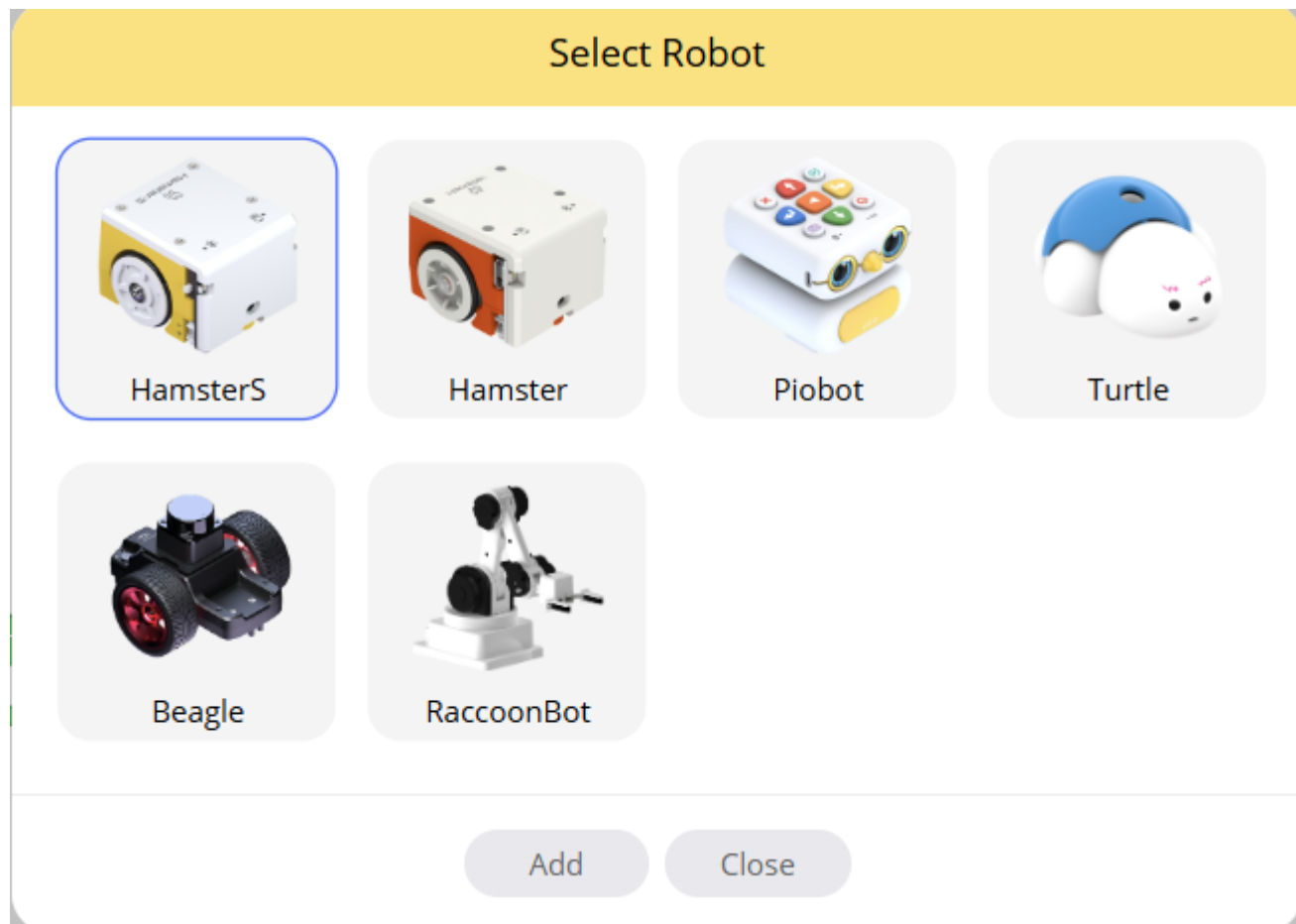
connecting again.

Robot



You can **select the robot to use** and register the robot's **information and dedicated blocks/scripts** in the program.

To control a robot in the program, the robot's information and blocks must first be added to the program.

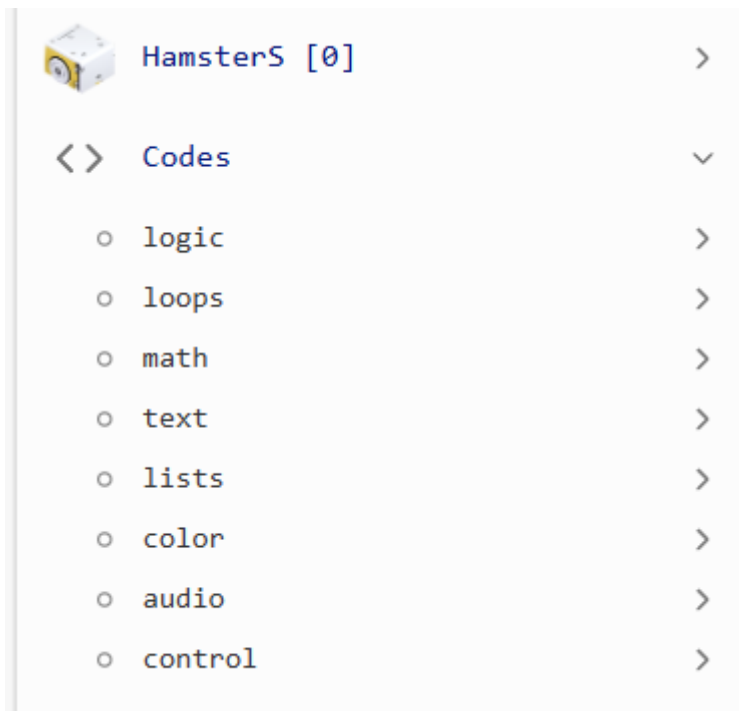


When you click the **Robot Selection** button, a popup window displays a list of robots available in the program.

- Hamster S

- Hamster
- PiBot
- Turtle
- Beagle
- RaccoonBot

After selecting the desired robot, click the **Add** button to register the robot's information and its dedicated blocks/scripts in the program.



When a robot is added, the following items are created:

- **Block Composer:** Dedicated **blocks** for the selected robot are added to the left **Block Palette**
- **Script Composer:** Dedicated **script code** for the selected robot is added to the left **Code List**

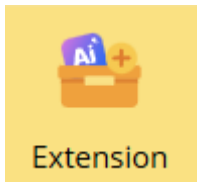
This allows you to freely control real robot hardware such as sensors, motors, and LEDs.

※ Reference

In RobomationLAB, you can connect and use as many robots as you want, regardless of the robot type or quantity.

However, if you want to connect and use multiple robots simultaneously: - The same number of dongles as the number of robots must be connected to the program - The same number of robots must be added to the program

Extension



Extension modules provide **AI-based extended features** such as voice recognition, computer vision, and image analysis.

You can **select the extension modules to use** and register the selected modules' **information and dedicated blocks/scripts** in the program.

◀ Back

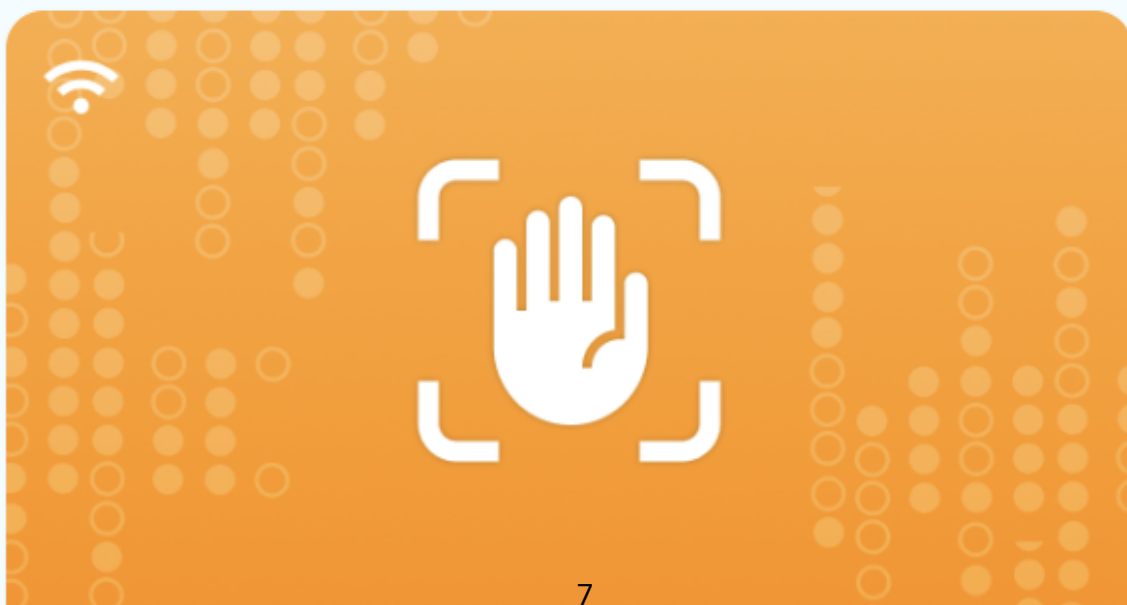
🔍 Search

All



1. Speech to Text

The icon features a blue background with a white Wi-Fi symbol in the top left corner. In the center, there is a white microphone icon with a speech bubble above it. A faint white line graph is visible in the background.



The icon features an orange background with a white Wi-Fi symbol in the top left corner. In the center, there is a white hand icon with fingers spread, enclosed in a white square frame with rounded corners. The background is decorated with a pattern of small white circles.

When you click the **Extension** button, a popup window appears showing the list of extension modules available in the program.

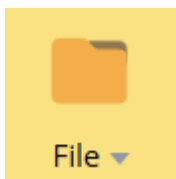
- Speech to Text
- Face Detection
- Detailed Face Detection
- Face Expression
- Hand Detection
- Body Detection
- Object Detection
- Color Detection

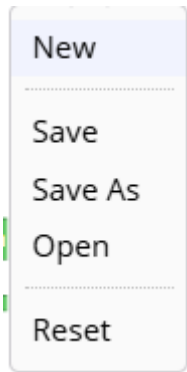
When you click a module, the module's information and its dedicated blocks/scripts are registered in the program, just like in **Robot**.

When you add camera-based extension modules such as **Face Detection** or **Hand Detection**, a camera module is created in the **Preview – Camera** tab, allowing you to connect and use a camera in the program.

If an extension module is no longer needed, you can remove it from the list by **right-clicking**  **Remove**.

File

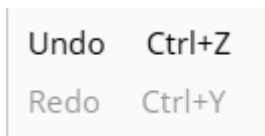
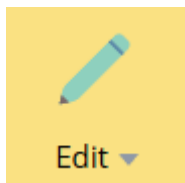




You can manage files such as creating new code, saving your work, and opening existing files.

- **New:** Clears the code in the active editor and creates a new file.
- **Save:** Saves the current code file. The file is saved in the user's **Downloads** folder with the **.block** file extension.
- **Save As:** Saves the code file under a new name. The file is saved in the user's **Downloads** folder with the **.block** file extension.
- **Open:** Opens a **.block** file from the user's computer.
- **Reset:** Resets all data registered in the program, including robot information, blocks/scripts, and settings. (robot information, blocks/scripts, settings, etc.).

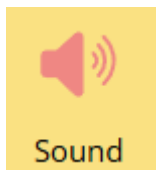
Edit



This menu allows you to undo or redo actions.

- **Undo (Ctrl+Z):** Cancels the most recent action.
- **Redo (Ctrl+Y):** Reapplies the action that was undone.

Sound



You can select sounds to use in coding or add your own offline sound files to the program.

Selecting Sounds

◀ Back

🔍 Search

AllAnimalsEffects



A Bass



A Elec Bass



Alert



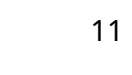
Alien Creak1



B Trombone



B Trumpet





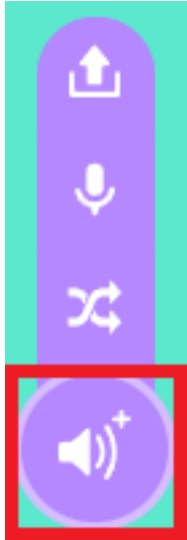
11

When you click the **Sound** button, a screen appears where you can choose from various sounds provided by the program.

You can use the following features:

- Search sounds
- ☐ Preview sounds
- Add sounds to the sound list (left panel)

Extended Features

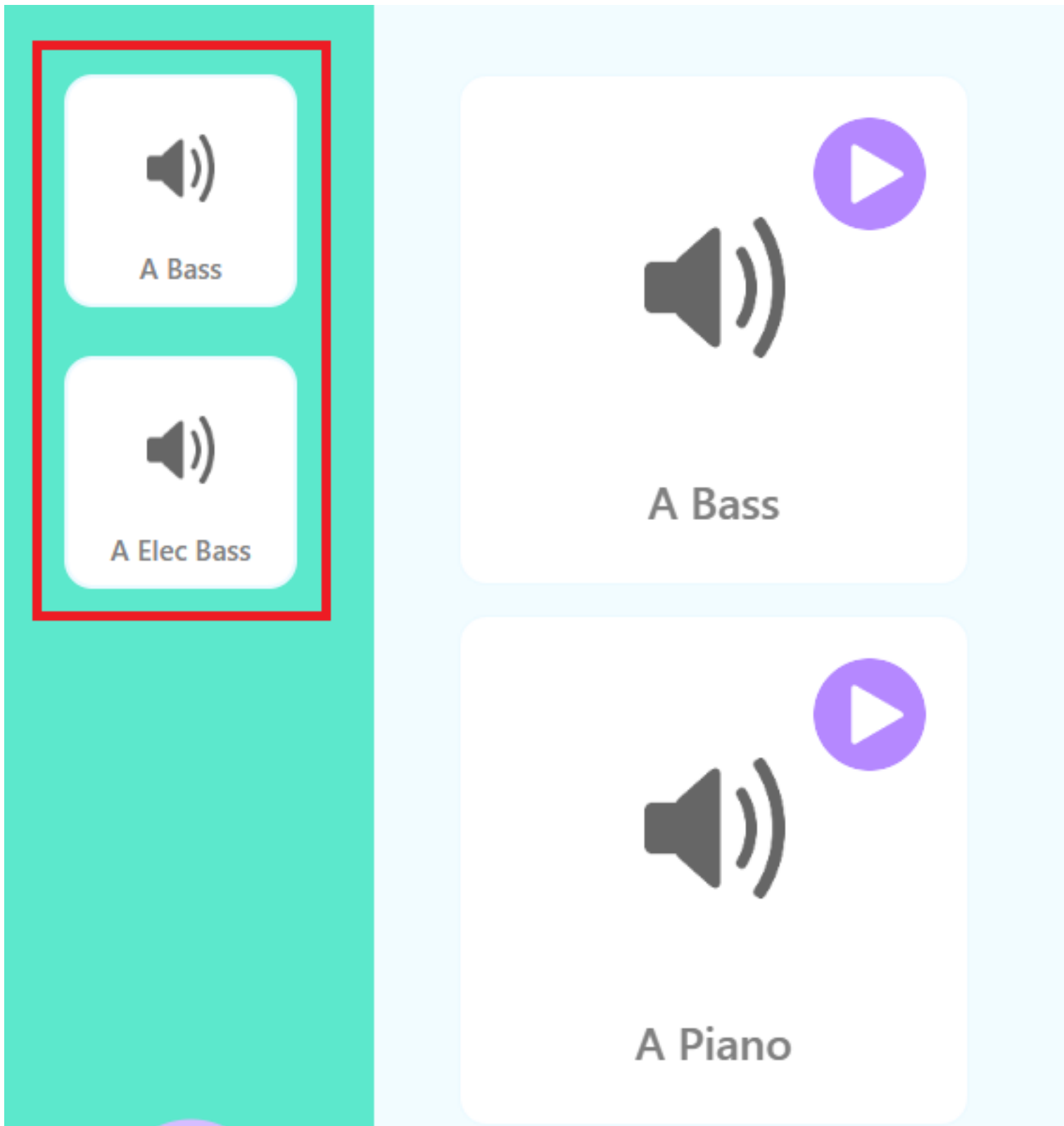


When you click the **Extension** button (red box) at the bottom-left, or hover over it, three extended feature options appear.

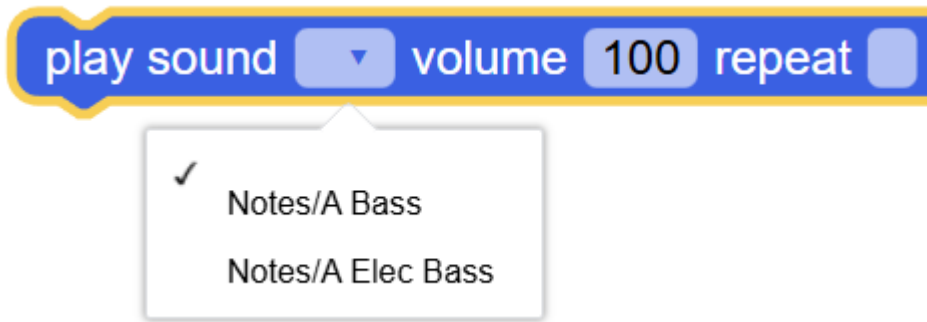
You can use the following features:

- **Add Local File:** Add an audio file from the user's computer
- **Record Sound:** Record and add your own sound
- **Add Random Sound:** Add a randomly selected sound from the entire sound list

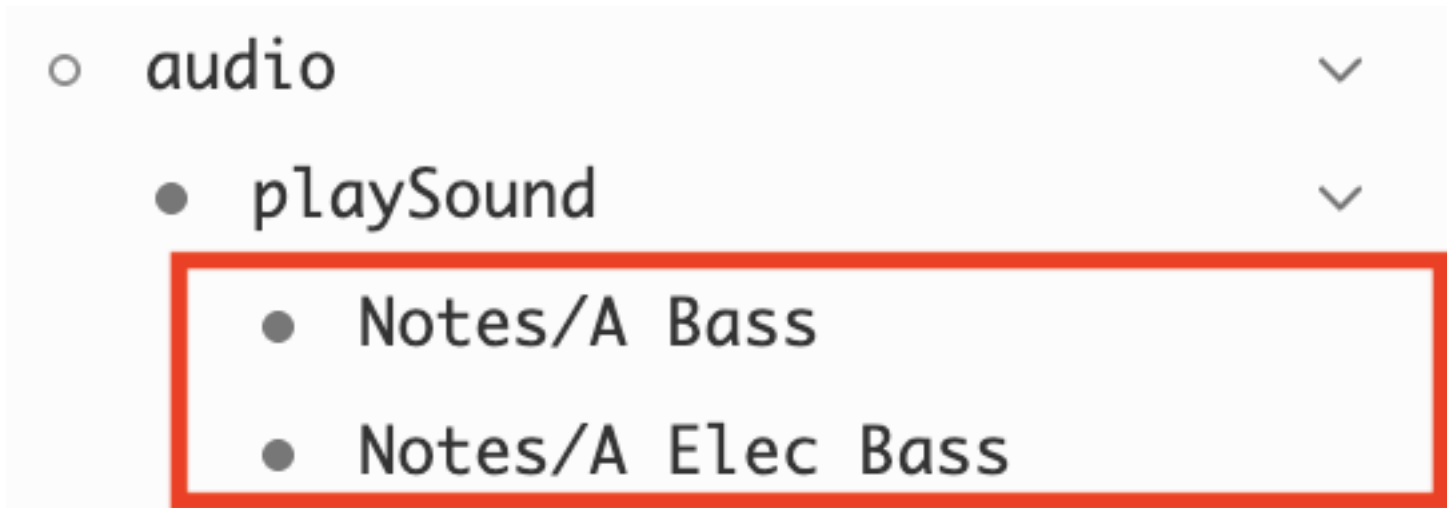
Using Sounds in Coding



Sounds added to the sound list (left panel) can be used in coding.



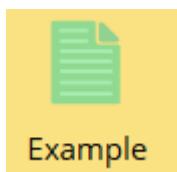
In **Block Coding**, you can select a desired sound from the dropdown menu of the **Play Sound** block.



In **Script Coding**, you can select a desired sound from the sub-options of the **playSound** function in the **Codes – audio** category.

When the code is executed, the selected sound is played through the user's computer speakers.

Example



When a robot is added to the program, you can load and try simple examples for each robot.

Selecting an Example

◀ Back

🔍 Search

All

Basic

Play



1. Speed Control



2. Acceleration Control



7. LED Dimming Control



8. Buzzer - Frequency Control



13. Floor Sensor



14. Fall Detect

When you click the **Examples** button, the **Select Example** screen appears as shown above. You can quickly find the desired example using the **category filter** and **search** functions.

Loading an Example

1. Click the **Examples** menu to open the **Select Example** screen, and choose the desired example.
2. The screen refreshes and the example appears in the coding area.
3. After loading the example, click the **Run** (▶) button to check how it works without any additional setup.

Run / Stop Code

Run (▶)



Interprets and executes the block code or script code written in the currently active editor.

Based on the written code, you can control the robots connected to the program.

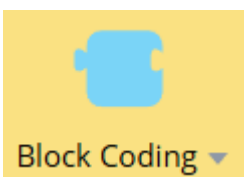
While the code is running, the code being written cannot be modified.

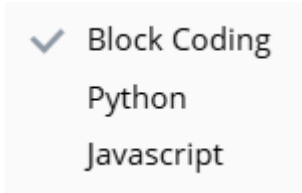
Stop (■)



Stops code execution.

Editor Settings





You can choose one of the **Block Coding, Python, or JavaScript** editors to write your code.

Even if you switch editors, any previously written code is preserved, allowing you to continue coding at any time.

※ **Each editor operates independently.**

Block Coding Editor

When Block Coding is selected, the logo changes to **Block Composer**.

In the Block Coding Editor, you can view real-time conversion of the block code into Python and JavaScript through the **Preview – Code** tab.

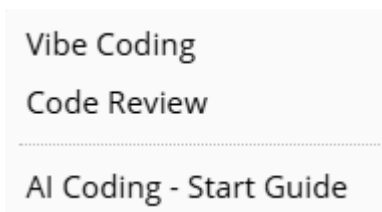
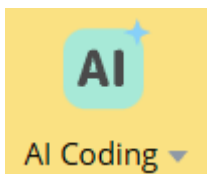
However, these converted codes are **not automatically applied** to the **Python** or **JavaScript** editors.

Python / JavaScript Editor

When Python or JavaScript is selected, the logo changes to **Script Composer**.

As with the Block Coding Editor, code written in the **Python** or **JavaScript** editor is not automatically reflected in the **Block Coding Editor** and is managed separately.

AI Coding

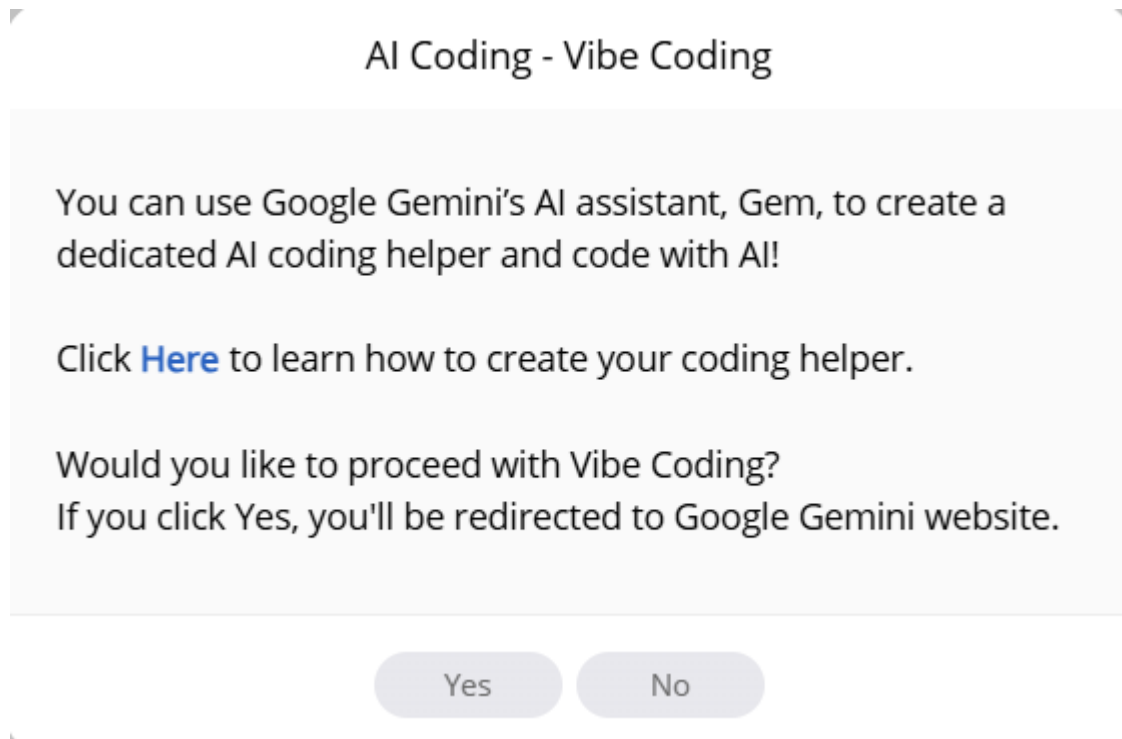


You can create your own **AI coding assistant** using **Gem**, an AI assistant powered by **Google Gemini**, and write code together with AI.

Vibe Coding

Vibe Coding is a new development approach that allows you to turn ideas or desired features into code using AI, even without advanced programming knowledge.

Through **collaboration with AI**, you can learn coding more easily.



Code Review

In **Code Review**, you can receive reviews and evaluations of your code from the AI coding assistant. Even when errors occur during code execution, you can ask the AI for the cause of the error and how to fix it to quickly resolve the issue.

AI Coding - Code Review

You can use Google Gemini's AI assistant, Gem, to create a dedicated AI coding helper and review your code!

Click [Here](#) to learn how to create your coding helper.

Your code and the review request message will be automatically copied to your clipboard. Paste (Ctrl+V) the copied message to view the code review results.

Would you like to check the review now?

If you click Yes, you'll be redirected to Google Gemini website.

Yes

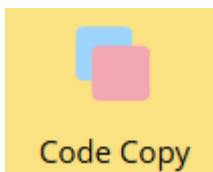
No

AI Coding – Getting Started Guide

This is a guide page for users who are new to the AI Coding feature.

It provides step-by-step instructions on **how to create and use an AI coding assistant**, as well as **how to utilize Vibe Coding and Code Review**.

Code Copy



You can copy the code written in the currently active editor to the clipboard.

Block Composer (Block Coding)

The copied result depends on the language selected in the **Preview - Code** tab. - Python selected □
Python code is copied

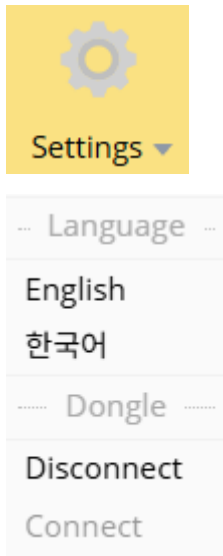
- JavaScript selected □ JavaScript code is copied

Script Composer (Python / JavaScript)

The code written in the editor is copied as-is.

You can paste the copied code wherever you want using **Ctrl+V**.

Settings



You can configure the basic settings of the program.

Language

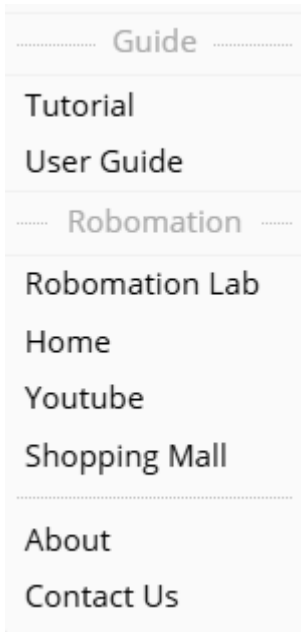
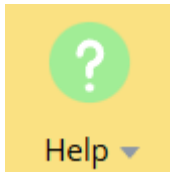
Changes the display language of the program.

The available languages are Korean and English.

Dongle

- **Disconnect:** Disconnects all dongles currently connected to the program.
- **Reconnect:** Reconnects the dongles to the program.

Help



You can access guides and external resources needed to use the program.

- **Tutorial:** Provides tutorials for users who are new to the program.
- **User Guide:** Provides detailed explanations of each component of the program, how to use them, and the functions and syntax of each block/script code provided by RobomationLAB.
- **RobomationLAB:** Takes you to the RobomationLAB main page.
- **Homepage:** Takes you to the official Robomation company website.
- **YouTube:** Takes you to the Robomation YouTube channel.
- **Shop:** Takes you to the Robomation online store.
- **Info:** Allows you to check the program version, update history, terms of use, and privacy policy.
- **Contact:** Allows you to submit inquiries about questions or bugs encountered while using the program.

Editor

The **Editor** is an area where you can write code to control a robot using **blocks** or **script code**.

Below, we introduce how to code in each environment—**Block Coding / Script Coding** (Python, JavaScript)—and the key points to keep in mind.

Block Coding Editor

Block Categories

The screenshot displays the Block Coding Editor interface. At the top, a yellow header bar contains the 'BLOCK COMPOSER' logo and several icons: a chain link (Dongle), a robot head (Robot), a folder with a plus sign (Extension), a folder (File), a pencil (Edit), a speaker (Sound), and a document (Example). Below the header, a sidebar on the left lists block categories, each with a colored circle icon: Logic (blue), Loops (green), Math (green), Text (yellow), Lists (purple), Color (purple), Audio (blue), Control (teal), Variables (orange), Functions (pink), and Others (grey). At the bottom of the sidebar is a 'HamsterS' category with a robot icon. The main workspace on the right shows a 'function setup' block containing three sub-blocks: 'HamsterS : set left wheel speed to 50', 'HamsterS : set right wheel speed to -50', and 'HamsterS : set both LED to basic color'.

BLOCK COMPOSER

Dongle Robot Extension File Edit Sound Example

Logic
Loops
Math
Text
Lists
Color
Audio
Control
Variables
Functions
Others

HamsterS

function setup

HamsterS : set left wheel speed to 50

HamsterS : set right wheel speed to -50

HamsterS : set both LED to basic color

This area organizes the blocks provided by RobomationLAB into **categories**.

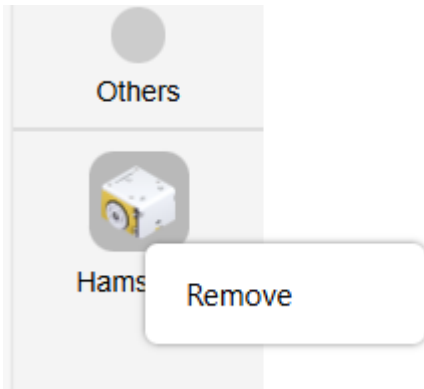
When you click a category, you can view the **block palette** corresponding to that category.

The following are the default block categories provided:

- Logic
- Loops
- Math
- Text
- Lists
- Color
- Sound
- Control
- Variables
- Functions
- Others

In addition, when you add a **Robot** or an **Extension Module** to the program, you can use dedicated block palettes for them within the program.

※ **Note**



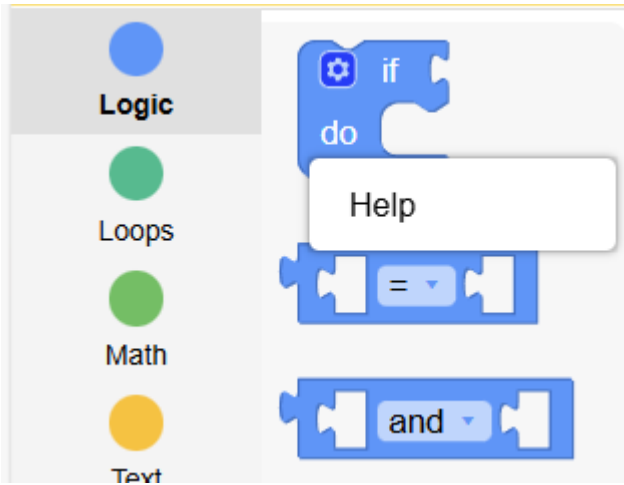
Among the blocks added to the program, any category that is no longer needed can be removed from the block categories by **Right-click** ▢ **Remove**.

Block Palette



This area contains all the blocks for each category.

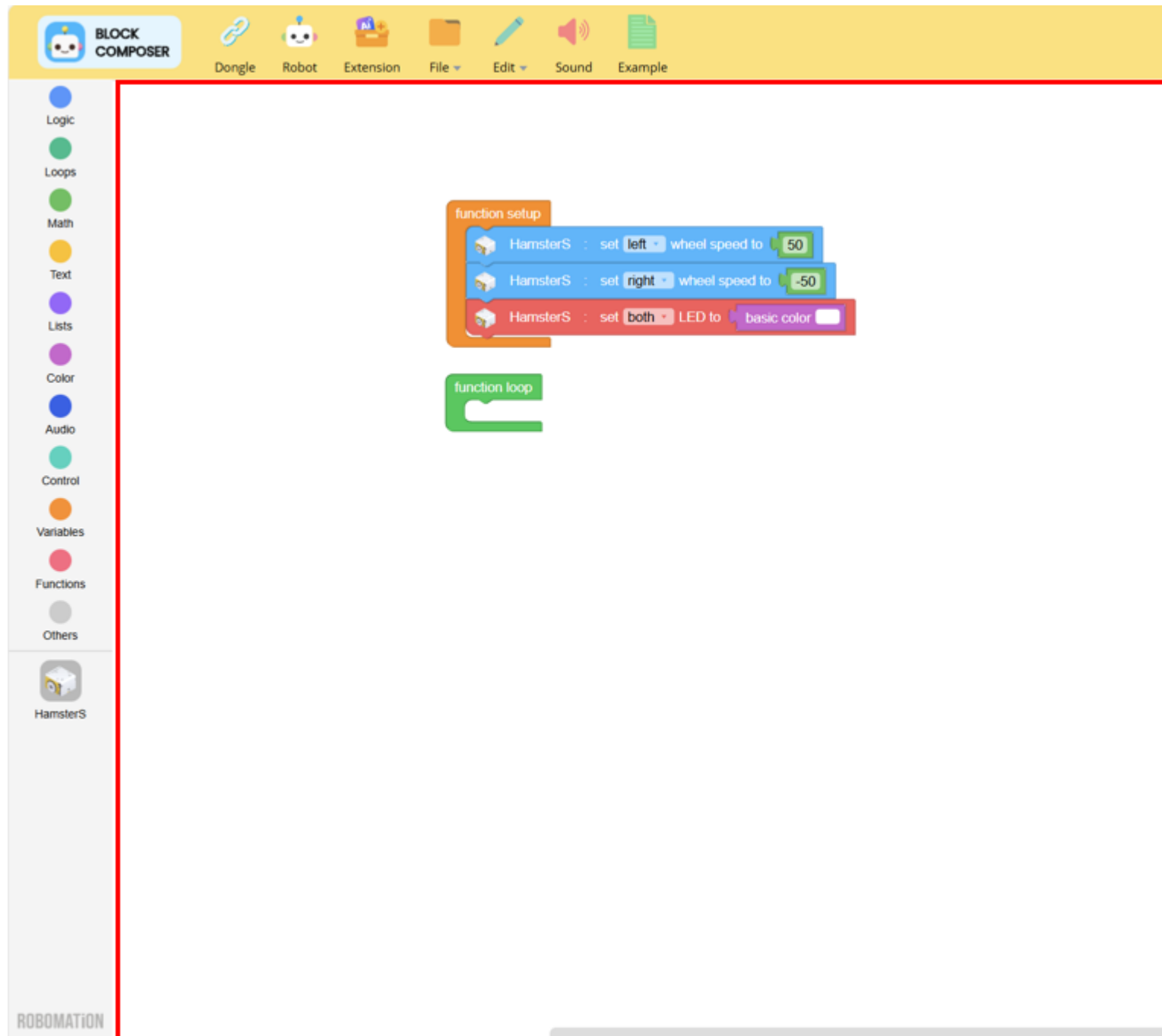
Blocks in the block palette can be moved to the coding area using **Drag & Drop**.



※ **Note**

If you want to check how to use a block, you can open the help guide for each block by **Right-click**  **Help**.

Coding Area



This is the area where you can assemble blocks taken from the block palette.

The assembled blocks are converted into **Python/JavaScript code** in real time.

When you run the code, it is interpreted to move and control the robot.

Basic Block Structure

When coding in the Block Coding Editor, you must follow the basic structure below.



In the Block Coding Editor, the program runs by interpreting the code inside the **function setup** and **function loop** blocks.

Therefore, you must place blocks inside the **function setup** and **function loop** blocks to write your code.

function setup

Inside the function setup block, define the actions to be performed initially when the code starts running. You can use the **wait** block to ensure actions are executed in sequence over time.

function loop

Inside the function loop block, define the actions to be repeatedly executed while the code is running. The defined actions are executed repeatedly every 10 ms.

Caution: Caution

Inside the **function loop** block, you cannot use blocks that include the **wait** function. Using them incorrectly may cause errors in your code.

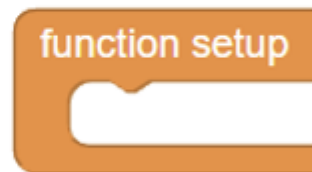
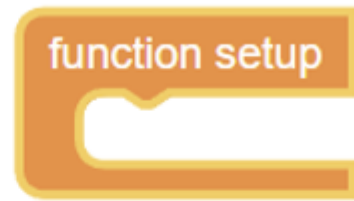
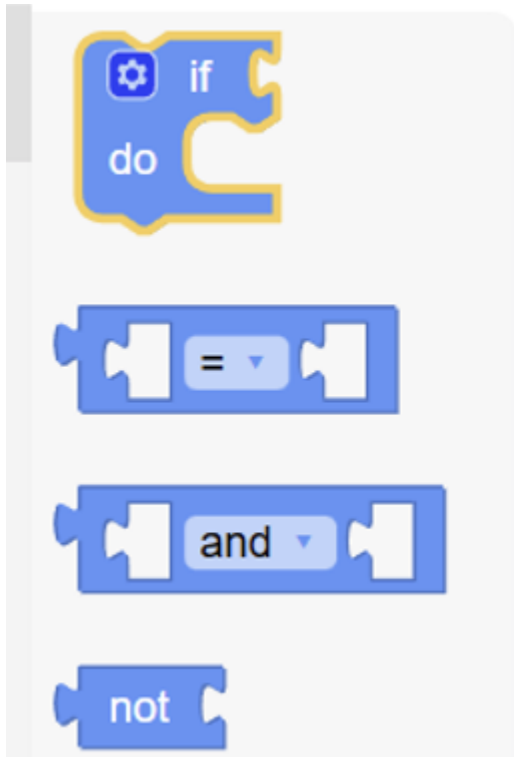
※ Note

(Except for custom functions created through the **Functions** category)

Blocks placed outside the **function setup** or **function loop** blocks have no effect when the code is executed.

How to Use Blocks

Adding Blocks





You can add a block by **dragging it from the block palette** and **dropping it into the editor**.

Copy / Paste Blocks

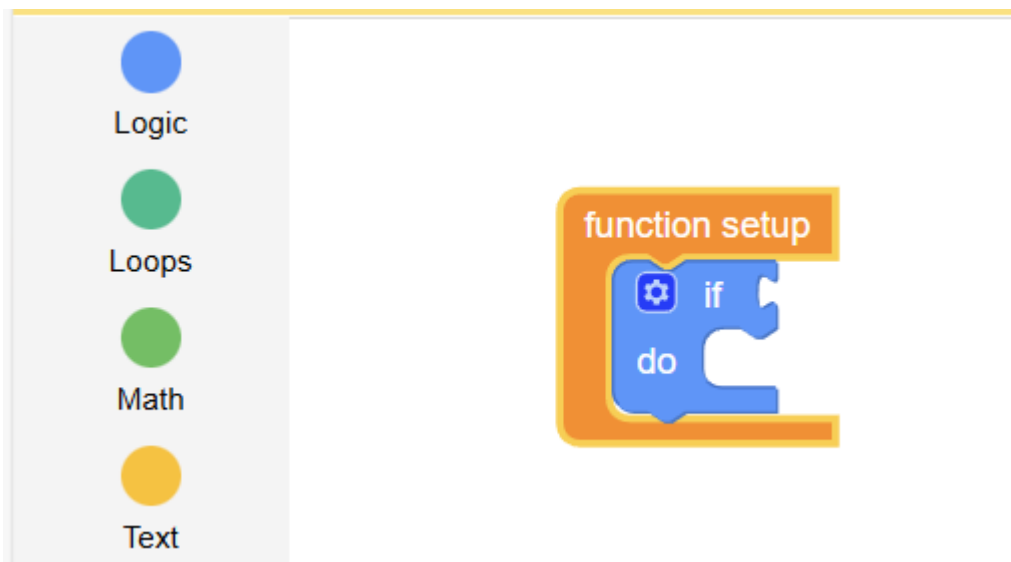
After selecting a block, press **Ctrl + C** to **copy** the selected block.

Press **Ctrl + V** to **paste** the most recently copied block into the editor.

Deleting Blocks

There are three ways to delete blocks in the editor.

1. Select a block and press the **Backspace** key to delete the selected block.
2. **Drag** the block you want to delete **from the editor** and **drop it into the block category** to remove it.





Logic



Loops



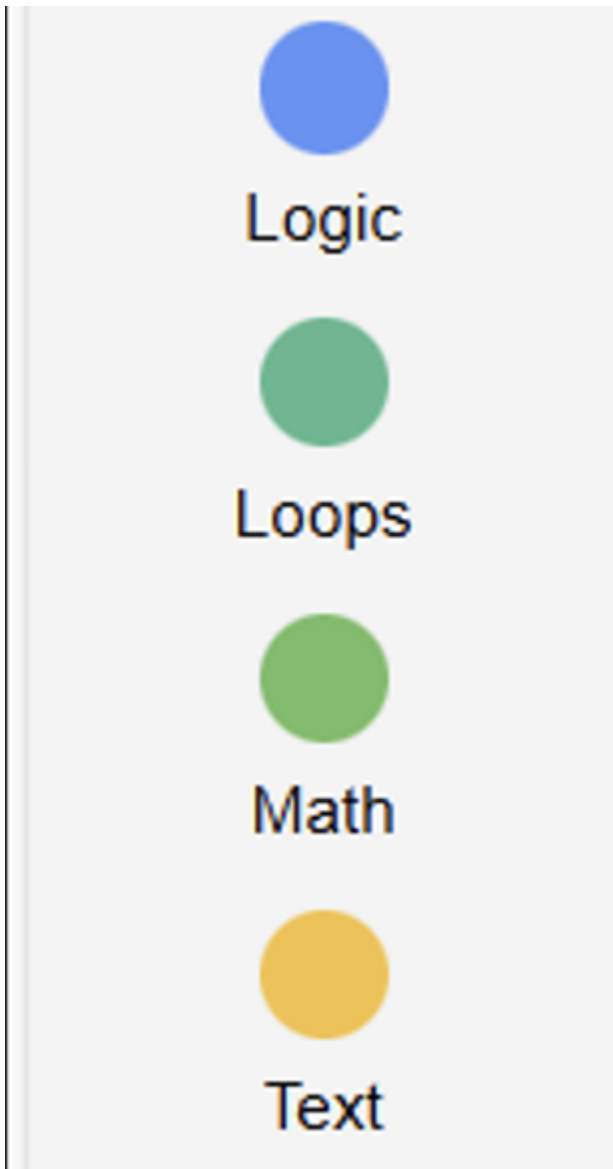
Math



Text



function setu



3. You can delete a block by **dragging it from the editor** and **dropping it into the trash bin**. Deleted blocks can be reviewed again in the **trash bin**.



Selecting Multiple Blocks at Once

While holding down the **Shift** key, drag on the screen or click multiple blocks to select multiple blocks at once and move, copy, or delete them together.

Additional Options

You can also **right-click** a block to access various additional options such as **Collapse / Expand Block, Enable / Disable, and Help**.

Script Editor

Code Collection

 **SCRIPT COMPOSER**

 Dongle

 Robot

 Extension

 File ▾

 Edit ▾

 Sound

Code Palette

 HamsterS [0] >

<> Codes ▾

o logic >

o loops >

o math >

o text >

o lists >

o color >

o audio >

o control >

Javascript Editor

<

1 // put setup code here,
2 - async function setup() {
3 \$('HamsterS*0:wheel.s
4 \$('HamsterS*0:wheel.s
5 \$('HamsterS*0:led.lef
6 \$('HamsterS*0:led.rig
7 }
8
9 // put control code here
10 - async function loop() {
11 }
12

This area organizes the **basic functions** required for robot coding and the **script code** dedicated to robots and extension modules into **categories**.

The following are the code categories provided in the basic functions (Codes):

- logic
- loops
- math
- text
- lists
- color
- audio
- control

The code provided in the basic functions performs the same roles as the basic blocks in the Block Coding Editor.

※ **Note**

You can learn how to use the code collection in the Script Coding Editor in **How to Use the Code Collection**.

Code Editor

 **SCRIPT COMPOSER**

 Dongle

 Robot

 Extension

 File ▾

 Edit ▾

 Sound

Code Palette

 HamsterS [0] >

<> Codes ▾

○ logic >

○ loops >

○ math >

○ text >

○ lists >

○ color >

○ audio >

○ control >

Python Editor

<

1 import asyncio

2

3 ▾ # put setup code here, t

4 ▾ async def setup():

5 __('HamsterS*0:wheel.

6 __('HamsterS*0:wheel.

7 __('HamsterS*0:led.le

8 __('HamsterS*0:led.ri

9 return

10

11 ▾ # put control code here,

12 ▾ async def loop():

13 return

14

This is the area where you can write code to control the robot.

Depending on the editor(Python / JavaScript) selected in **Editor Settings**, you can write code in your preferred programming language.

Basic Code Structure

When coding in the Code Editor, you must follow the basic structure below.

Python Editor

<

1 `import` `asyncio`

2

3 `# put setup code here, to run once:`

4 `async def` `setup()`:

5 `return`

6

7 `# put control code here, to run repeatedly:`

8 `async def` `loop()`:

9 `return`

10

Javascript Editor

<

1 `// put setup code here, to run once:`

2 `async function` `setup()` {

3 }

4

5 `// put control code here, to run repeatedly:`

6 `async function` `loop()` {

7 }

8

In the Code Editor, the program runs by interpreting the code inside the **setup** and **loop** functions. Therefore, you must write your code inside the **setup** and **loop** functions.

setup

Inside the setup function, define the actions to be performed initially when the code starts running. You can use the **wait** function to ensure actions are executed in sequence over time.

loop

Inside the loop function, define the actions to be repeatedly executed while the code is running. The defined actions are executed repeatedly every 10 ms.

Caution: Caution

Inside the **loop** function, you cannot use functions that require async/await functionality, such as the **wait** function.

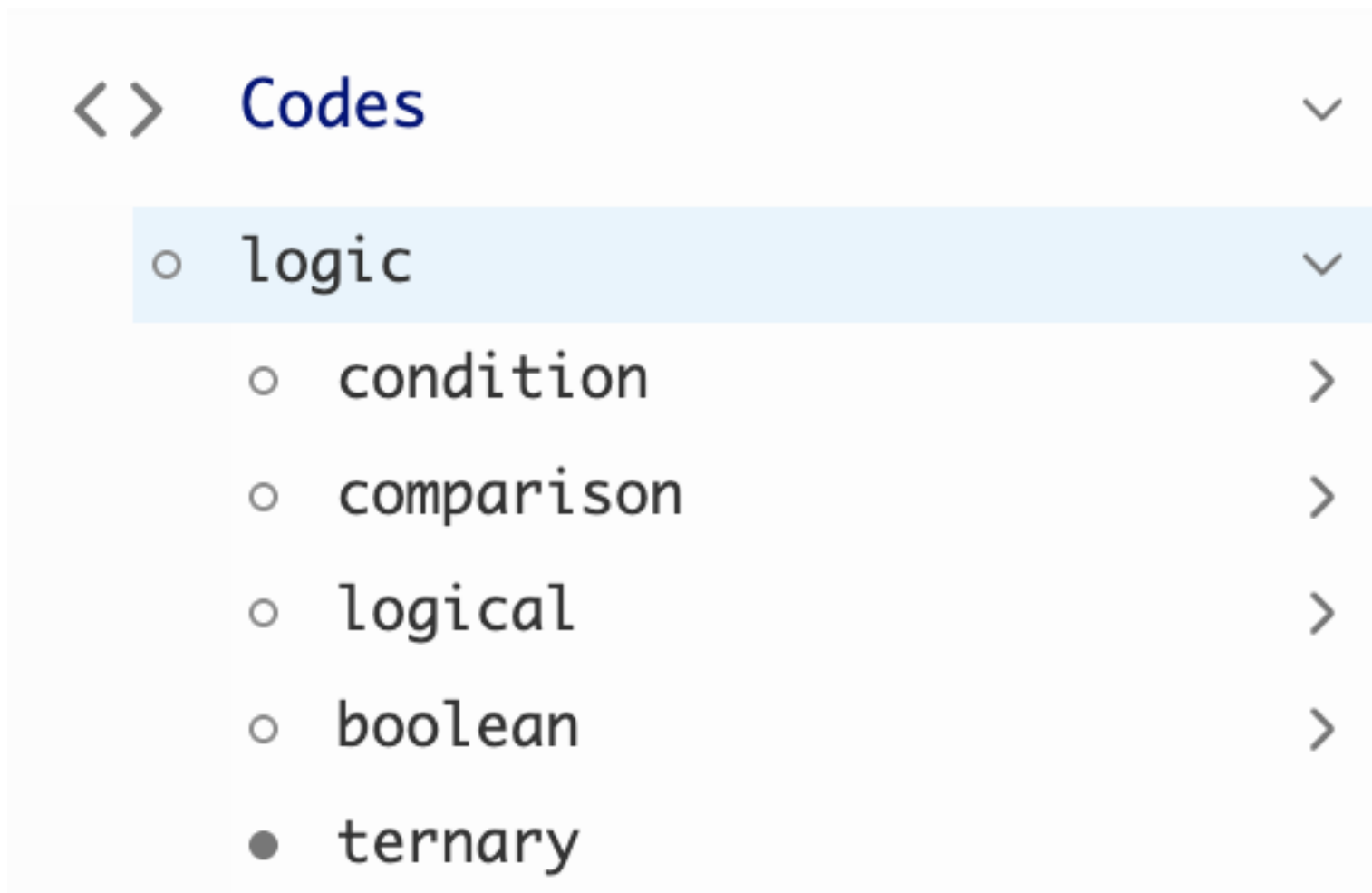
If used incorrectly, this may cause errors in your code.

How to Use the Code Collection

Below, we explain **how to use the code collection in your coding** with a simple example.

Finding the Code You Want

The code collection provides various functions and code needed for robot coding.



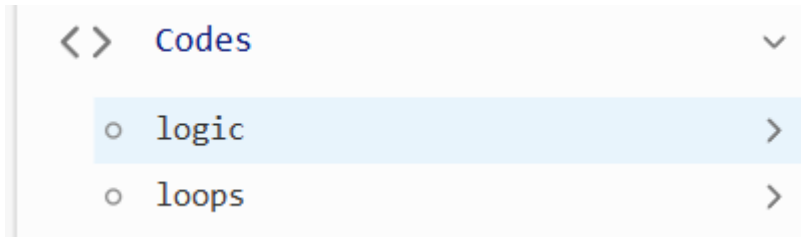
When you check the menus under the **Codes** category, you can find menus that have the icon.

When you click the **logic** menu once, the icon changes to , allowing you to expand and view the **sub-menus** inside.

Menus that contain submenus in this way are called **categories**.

If a menu in the **logic** category, such as **ternary**, does not have the icon, it means that there are no more **submenus** within that menu.

Menus that do not have submenus like this are called **codes**.



When you click a category with expanded submenus again, the icon changes back to  and the submenus are hidden.

By following categories in this way, you can find the code you want in the code collection.

Inserting Code into the Editor

The following explains how to insert code into the editor.



Codes



- logic



- condition



- comparison



- logical



- boolean



- ternary

If a menu has the icon to the left of its name,

it means that the menu does not have any selectable **code options**.

Most menus that fall into this category are **category** menus that contain submenus.

If a menu has the icon to the left of its name, it means that the menu allows you to select **code options**.

Most menus that fall into this category are **code** menus that do not have submenus.

If you **right-click** the code you want to insert into the editor,

you can view the available options.



Codes



- logic



- condition



- if

- else if

- else

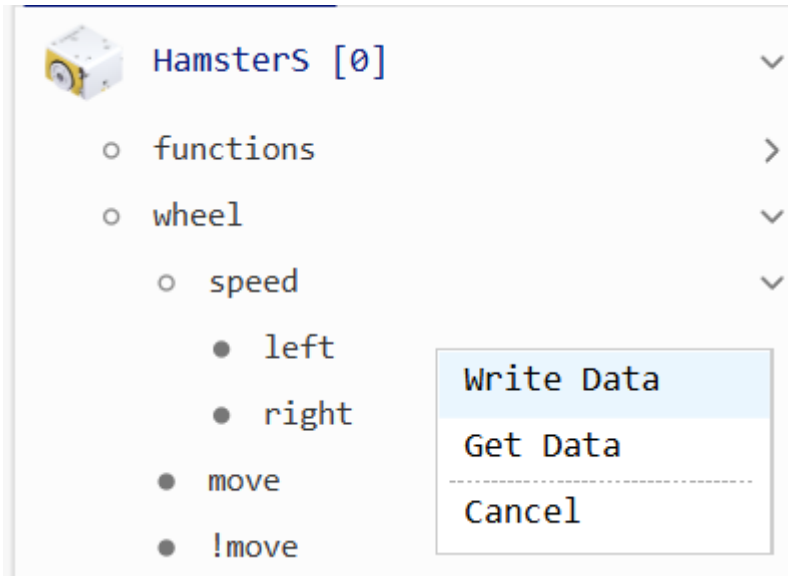
Insert Code

Cancel

Codes in the **Basic Functions** category can have the following options (e.g., the **if** code in the **logic** category):

- **Insert Code**

Inserts a predefined function or code into the editor.



Codes in the **Robot Code** category can have the following options (e.g., the **wheel.speed.left** code in the **Hamster S** category):

- **Insert Code**

Inserts a predefined function or code into the editor.

This option is most commonly seen when inserting code from the **custom functions** (functions) category provided for each robot.

- **Get Value**

Inserts code into the editor that allows you to check values such as the robot's wheel speed, LED color (user-settable values), or sensor values.

```
# Left wheel speed value of Hamster S
__('HamsterS*0:wheel.speed.left').d
```

- **Set Value**

Inserts code into the editor that is used when you need to write values to control the robot, such as setting the wheel speed or specifying a movement distance.

```
# Set the left wheel speed of Hamster S to 50
__('HamsterS*0:wheel.speed.left').d = 50
```


- **Wait**

Inserts code into the editor that is used to control the robot's actions in sequence over time, such as waiting until movement is completed after setting a movement distance.

This option can usually be found in 'Event-type' codes that have a ! at the beginning of their name.

```
# Hamster S moves 5 cm and waits until the movement is completed.
__( 'HamsterS*0:wheel.move').d = __getDistance('HamsterS', 5, 'cm')
await __('HamsterS*0:wheel.!move').w()
```

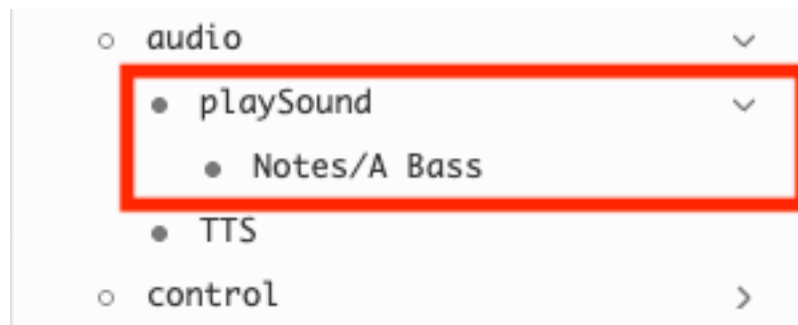
- **Check Event**

Inserts code into the editor that allows you to check whether a specific event has occurred, such as whether the robot's movement has been completed or whether a tap event has been detected.

This option can usually be found in **Event-type** codes that have a ! at the beginning of their name.

```
if __('HamsterS*0:wheel.!move').e == True:
    print("Event")
```

Codes with Constant Values



Among the code menus, there are menus that both **contain submenus and allow code insertion**, as shown below

(e.g., the **playSound** code in the **audio** category).

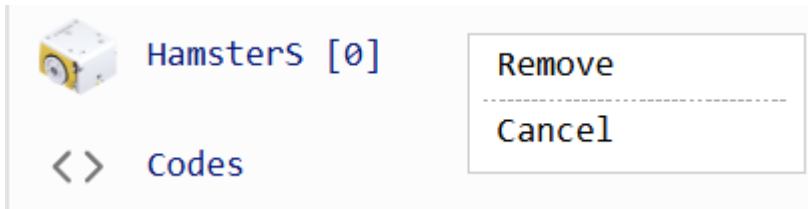
These codes are referred to as **codes with constant values**, and the codes in the submenus of these menus are called **constant values**.

For these codes,

when you click the menu to expand the **submenus**, you can view the constant values that are predefined as input values.

You can also **right-click** to select **code options** and insert the code into the editor.

※ Note

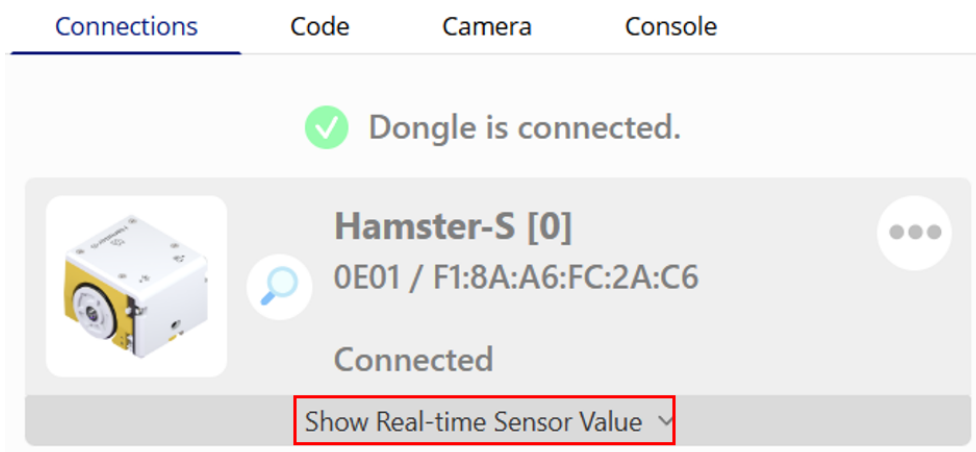


Among the robot- or extension module-specific code added to the program, any code category that is no longer needed can be removed from the code collection by **Right-click** **Remove**.

Preview

Preview is an area that helps with robot coding by allowing you to check the **robot connection status** and by providing tools such as the code console and camera console. Below, the functions available in each tab are described.

Robot Connection Status Tab



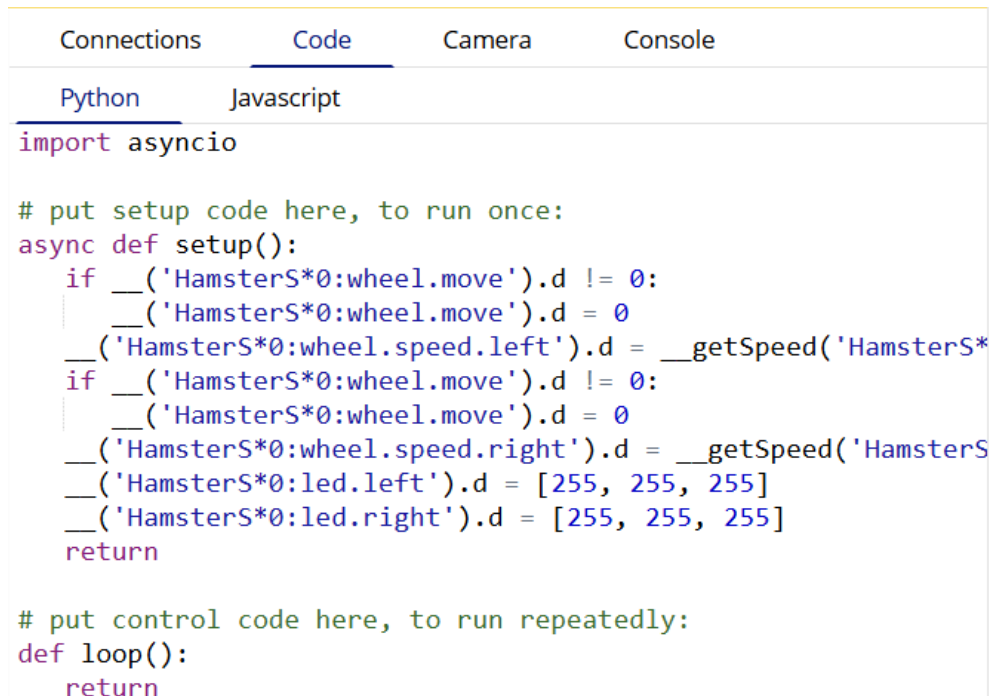
In the Robot Connection Status tab, you can check information about the robots that are currently connected.

By clicking **Hide Real-Time Sensor Values**, you can monitor the robot's sensor data in real time, displayed as graphs or numerical values.



1. You can select which sensor to view, and the types of sensors supported vary by robot.
Depending on the selected sensor, the graph and data display format automatically adjust to match that sensor's values.
2. You can choose the time interval (in seconds) at which sensor values are checked or recorded.
Based on the selected interval, the maximum duration for which data can be recorded and displayed on the graph will vary.
3. When you place the mouse cursor over the sensor graph, you can view the sensor value at that specific point in time in real time.
4. Start / Stop Recording: Records sensor data.
5. Export: Saves the recorded sensor data as a .csv file and the graph view as a .png file.
This feature is only available after data has been recorded.
6. Capture: Saves the current graph view as a .png file in real time.
You can record changes in sensor values and save both the recorded data files and the graph images as files.

Code Tab



The screenshot shows the 'Code' tab in a software interface. At the top, there are four tabs: 'Connections', 'Code' (which is selected and underlined), 'Camera', and 'Console'. Below these, there are two sub-tabs: 'Python' (selected and underlined) and 'Javascript'. The main area contains Python code for an asynchronous setup and a loop function. The code is color-coded: keywords like 'import', 'def', 'if', 'return', and 'async' are in purple; comments are in green; and strings and function names are in blue.

```
import asyncio

# put setup code here, to run once:
async def setup():
    if __('HamsterS*0:wheel.move').d != 0:
        __('HamsterS*0:wheel.move').d = 0
    __('HamsterS*0:wheel.speed.left').d = __getSpeed('HamsterS*
    if __('HamsterS*0:wheel.move').d != 0:
        __('HamsterS*0:wheel.move').d = 0
    __('HamsterS*0:wheel.speed.right').d = __getSpeed('HamsterS
    __('HamsterS*0:led.left').d = [255, 255, 255]
    __('HamsterS*0:led.right').d = [255, 255, 255]
    return

# put control code here, to run repeatedly:
def loop():
    return
```

In the Code tab, you can see how adding or removing blocks is reflected in **Python** and **JavaScript** code in real time.

By observing how blocks are converted into code, you can more easily learn and understand coding syntax.

※ The Code tab is available **only in the Block Coding Editor**.

In the Python / JavaScript editors, users write code directly, so a separate Code tab is not provided.

Camera Tab



In the Camera tab, you can view the live video feed from a camera connected to your PC in real time.

This feature is enabled **only when a camera-based extension module is added to the program**, and a corresponding camera module will appear in the Camera tab.

Each camera module provides the following functions:

- Select one of the cameras available on the current PC and display its video feed.
- Use the **On / Off** button to show or stop the camera view.

Console Tab

In the Console tab, you can monitor log messages (Log) and scope graphs (Scope) in real time while the program is running.

This tab is useful for debugging, sensor value monitoring, and graph-based analysis.

Log

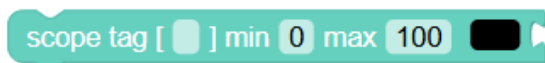
By using the **Log Output** block, you can print text or numeric values to the console log area together with a specified tag.



Log	Scope
[Internal Sensors]	Temp0 °C Battery :0 V Signal :-25 dBm
[Internal Sensors]	Temp23.5 °C Battery :3.6 V Signal :-24 dBm
[Internal Sensors]	Temp23.5 °C Battery :3.6 V Signal :-24 dBm
[Internal Sensors]	Temp23.5 °C Battery :3.6 V Signal :-25 dBm
[Internal Sensors]	Temp23.5 °C Battery :3.6 V Signal :-23 dBm
[Internal Sensors]	Temp23.5 °C Battery :3.6 V Signal :-26 dBm
[Internal Sensors]	Temp23.5 °C Battery :3.6 V Signal :-25 dBm
[Internal Sensors]	Temp23.5 °C Battery :3.6 V Signal :-27 dBm
[Internal Sensors]	Temp23.5 °C Battery :3.6 V Signal :-26 dBm
[Internal Sensors]	Temp23.5 °C Battery :3.6 V Signal :-24 dBm
[Internal Sensors]	Temp23.5 °C Battery :3.6 V Signal :-26 dBm
[Internal Sensors]	Temp23.5 °C Battery :3.6 V Signal :-23 dBm
[Internal Sensors]	Temp23.5 °C Battery :3.6 V Signal :-25 dBm
[Internal Sensors]	Temp23.5 °C Battery :3.6 V Signal :-24 dBm
[Internal Sensors]	Temp23.5 °C Battery :3.6 V Signal :-25 dBm

Scope

By using the **Scope Output** block, you can visualize numeric values as **real-time graphs** for each tag. You can configure the minimum value, maximum value, and graph color to visually monitor changes in the desired data.



Log

Scope

Digital Scope[1] : proximity sensor

